YUVA INTERN

EMBEDDED SYSTEMS INTERNSHIP | WEEK 3 TECHNICAL REPORT

IoT-Based Smart Environmental Monitoring and Alert System

Finding and fixing defects, and measuring and reducing firmware cost, on a simulated embedded system

Week 3 Task

Debugging and Performance Optimization

Item	Details
Student Name	Pilla Yashwanth (Yuva Intern)
Email	yaswanthvasu79@gmail.com
Institute	Ignite embedded system
Internship Organization	Yuva Intern
Report Date	21/10/2026
Deliverables	smart_env_monitor.c v3.0.0 (single file), README.md, logs, analysis tools, this report?
Test Platform	Simulated hardware on a PC (GCC, Valgrind, cppcheck, gcov). No physical hardware was used.

Table of Contents

1. Introduction and Scope 3
1.1 Objectives 3
2. Debugging Methodology 3
3. Baseline Findings (Week 2 Firmware) 3
3.1 The Most Instructive Finding: D1 4
4. Defect Log 4
5. How Good Are the Tests? Coverage and Mutation Testing 4
6. Performance Analysis 5
7. Optimisations 6
7.1 Behaviour Unchanged 6
8. Results 6
9. Verification Summary 7
10. Limitations 7
11. Preparation for Week 4 8
12. Conclusion 8
Appendix A: Self-Test Output (final run) 9
Appendix B: Soak Test Report (168 simulated hours) 9
Appendix C: Benchmark Output, One Simulated Hour 9
Appendix D: Valgrind on the Week 2 Baseline (excerpt) 9

1. Introduction and Scope

Week 2 delivered working firmware that passed its own tests. Week 3 asks a different question: **what is still wrong with it, and what does it cost to run?** The work has two parts. First, a systematic search for defects using tools that the Week 2 tests did not use (static analysis, Valgrind, coverage measurement, fuzzing, long randomised soak tests and mutation testing). Second, profiling of the firmware, followed by optimisations that are verified to leave behaviour unchanged.

1.1 Objectives

- Find defects that the Week 2 test suite missed, and fix them with regression tests.
- Measure how good the tests themselves are (coverage and mutation testing).
- Profile the firmware and optimise the largest costs, proving that behaviour is unchanged.
- Record every finding, including the ones that turned out to be weaknesses of my own tests.

2. Debugging Methodology

The Week 2 suite had 20 tests and passed. Passing tests only show that the tested paths work, so Week 3 used several independent techniques, each able to find a different kind of defect.

Table 1: Techniques used and what each can find

Technique	Tool	What it finds
Compiler warnings	GCC -Wall -Wextra -Wpedantic -Wshadow -Wconversion, at -O0, -O2, -O3, -Os	Type, format and truncation problems; behaviour that depends on optimisation level.
Static analysis	cppcheck 2.13	Scope, const-correctness and declaration mismatches.
Memory and UB checking	AddressSanitizer, UBSan, Valgrind memcheck	Invalid accesses, undefined behaviour, use of uninitialised memory.
Coverage	gcov (line and branch)	Firmware paths that no test ever executes.
Targeted probing	Bug-hunt harness (baseline v2)	Long-uptime and corner-case behaviour: clock rollover, threshold changes under alert.
Fuzzing	400000 random bytes into the UART interrupt	Crashes and broken invariants in the command parser.
Soak testing	Random faults and environment for up to 7 simulated days, 11 invariants checked every 50 ms	Rare interactions between faults, resets and state.
Mutation testing	16 deliberately injected defects	Whether the tests would notice a real defect (test quality).
Profiling	Valgrind callgrind (instruction counts)	Where the firmware spends its work, deterministically.

3. Baseline Findings (Week 2 Firmware)

Table 2: Results of each technique on the Week 2 baseline

Technique	Result	Meaning
GCC warnings / sanitizers	Clean at -O2; ASan and UBSan silent	Looked healthy.
cppcheck	Style findings (variable scope, parameter names, const pointers)	Minor (D4).
-Wconversion	1 narrowing conversion	Minor (D4).
Valgrind memcheck	**12606 errors in 101 contexts** (conditional jump on uninitialised value)	Real defect D1, invisible to the sanitizers.

Technique	Result	Meaning
Same tests at -O0	**19 of 20 passed**: the display was rewritten every second (30 writes instead of 6)	D1 shows up as a build-dependent failure.
gcov line coverage	94.9% (357 of 376 firmware lines)	Untested: display failure mid-run, 3 of 4 SET commands, HELP, MUTE OFF.
Bug hunt: clock rollover	State fell back to WARMUP and air status to WARMING UP after the tick wrapped	Real defect D2.
Bug hunt: SET during alert	Active alert was silenced although the condition still held	Real defect D3.
Fuzzing, worst-case payload	400000 random bytes survived; worst telemetry message 188 of 256 bytes	No defect.
Soak test, 6 simulated hours	0 invariant violations	No defect found at this length.

3.1 The Most Instructive Finding: D1

The display task decided whether the screen had changed by comparing two six-line text frames with `memcmp`. The frames were built with `snprintf`, which writes the text and a terminating zero but leaves the remaining bytes of each 22-byte line **uninitialised**. The comparison therefore included undefined padding. At -O2 the leftover stack contents happened to be identical every time, so the Week 2 tests passed (6 writes in 30 s). At -O0 they differed, and the same code rewrote the display every second (30 writes). Valgrind reported it immediately; AddressSanitizer and UBSan did not, because reading uninitialised memory is not an error for them. The lesson is that a test passing in one build is not proof that the code is correct.

4. Defect Log

Table 3: Defects found and fixed in Week 3

ID	Symptom	Root cause	Fix	Regression test
D1	Display rewritten every second at -O0; Valgrind: 12606 errors	memcmp over uninitialised frame padding	Frame buffer zero-filled before formatting	T26, T20
D2	After 49.7 days the state returns to WARMUP and the air alert is suppressed for 10 s; deadlines fire early	32-bit tick wrap: absolute warm-up compare and unsafe `>=` deadline comparisons	Signed-difference comparison `time_reached()`; warm-up latched once	T21, T22
D3	SET silenced an active alert although the value still exceeded the new limit	SET re-initialised the alert channel	`ch_set_limits()` changes limits only; state kept	T23
D4	Static-analysis and conversion warnings	Scope, const, parameter-name and type issues	Cleaned up	Build at 4 optimisation levels, 0 warnings
D5	Fault cleared late in a loop pass updated state and LED one pass (1 ms) late; found only by the 7-day soak, once in 168 h	Dirty evaluation ran before display/Wi-Fi tasks, which can change faults	Second dirty check at the end of the pass	T31 and soak

D2 is the kind of defect that only appears in the field. A device left running for 49.7 days would briefly return to its warm-up state, suppress the air alert and show WARMING UP on the display. It cannot be found by short tests, which is why a time-warp helper (`sim_time_warp`) was added to the simulator.

5. How Good Are the Tests? Coverage and Mutation Testing

Coverage shows what the tests execute; mutation testing shows whether they would notice a bug. In mutation testing a known defect is deliberately injected into a copy of the firmware, and the test suite is run. A test suite that still passes has a blind spot.

Sixteen mutants were tested. At first **six survived**, which exposed weaknesses in my own new tests. They were strengthened, and then all sixteen were caught.

Table 4: Mutants that initially survived, and how the tests were strengthened

Injected defect	Why the test missed it	Strengthening
SET resets the alert channel (D3)	With event-driven outputs, the LED stayed on for seconds from stale state, hiding the bug	Check the channel state immediately and that the LED never toggles
Mute change not applied immediately	Next blink toggle masked it within 250 ms	Check the buzzer silences within 3 ms
Missing re-evaluation when a fault is set or cleared	The blink timer caught up within 1 s	Check the state in the same loop pass the fault is declared
Spike filter removed	The spike was gone again before the test looked	Check the snapshot immediately after the spike sample
Hysteresis removed	Mutation applied to the wrong function in my first attempt	Mutation corrected; T04 and T06 catch it
Confirmation count ignored	Alert raised and cleared again within the test window	Check that the LED and buzzer never switched on at all

Table 5: Final mutation results

Injected defect	Caught by
M1 re-introduce D1 (frame not zero-filled)	T26
M2 re-introduce D2 (plain >= deadline compare)	T22
M3 re-introduce D3 (SET resets alert state)	T23
M4 ISR does extra work (sets a flag)	T27
M5 missing dirty on alert raise (O2 gating)	T03,T04,T06,T07,T12,T16,T17,T18,T21,T23
M6 missing dirty on mute change	T16
M7 scheduler drift	T22,T28
M8 DHT alerts not cleared on sensor fault	T31
M9 re-introduce D2 (absolute warm-up compare)	T21,T22
M10 missing dirty on fault set/clear	T09
M11 spike filter removed	T19
M12 OLED probe never written	T20,T24
M13 hysteresis removed (clear at trip point)	T04,T06
M14 confirmation count ignored	T05
M15 MQ-135 rail values accepted as valid	T10
M16 Wi-Fi back-off never grows	T12

Coverage of the firmware code rose from **94.9% (357/376 lines) to 99.3% (404/407 lines)**. The three remaining lines are a per-line display write-failure branch and a defensive branch for an over-long telemetry message. The latter cannot be reached with the current buffer: the worst-case message is 188 of 256 bytes (test T30).

6. Performance Analysis

A deterministic benchmark scenario was used for all measurements: 24 C base temperature with slow variation, a high-temperature alert episode (200 to 260 s), a Wi-Fi outage (300 to 330 s) and slowly varying humidity. It was profiled with callgrind for 10 simulated minutes, and run for one simulated hour to count display traffic.

Table 6: Baseline profile (Week 2, no function inlining so costs can be attributed, 10 min)

Function	Instructions	Share	Observation
state_update()	36.4 M	24.9%	Recomputed on **every** millisecond, though state changes only a few times per hour.
timer_isr_1ms()	20.4 M	14.0%	Six counters and flags per interrupt, 1000 times per second.
take_flag()	12.0 M	8.2%	Flag read and clear, per loop pass per task.

Function	Instructions	Share	Observation
fw_loop()	11.4 M	7.8%	Dispatch.
alert_task()	9.5 M	6.5%	Recomputed every millisecond.
wifi_task() + hal_wifi_connected()	15.4 M	10.5%	Link status polled every millisecond.
cli_poll()	8.4 M	5.7%	Function call per pass to find an empty buffer.
hal_wdt_feed() + warm_done()	6.4 M	4.4%	Watchdog fed every millisecond.
Total (incl. harness)	146.3 M	100%	

Two further costs were measured by counting simulated I2C bytes. With the display writing the full 1024-byte frame whenever any value changed, one simulated hour produced **1,753,088 bytes**, about 39.4 s of bus time at 400 kHz (1.1% of the time). At -O0, where defect D1 shows, it was 3,686,400 bytes.

7. Optimisations

Table 7: Optimisations implemented in Week 3

ID	Change	Rationale	Risk and control
O1	Timer ISR only advances the tick. Periodic tasks run from deadlines checked in the main loop (`due()`), drift-free and wrap-safe	Smaller ISR; removes all interrupt-shared task flags and the race they implied; fixes D2 in the same step	Task timing could change. T28 checks exact 2 s, 1 s and 500 ms periods; T27 checks the ISR changes nothing but the tick
O2	State and outputs re-evaluated only when something changed (dirty flag)	state_update and alert_task were 31% of the work and idle almost always	A missing trigger would leave stale outputs. Controlled by mutants M5, M6, M10, by 11 soak invariants and by the behaviour fingerprint
O3	Display: only changed lines are written (128 bytes), a 5 s one-line probe still detects a dead display, full redraw every 60 s	Cuts I2C traffic; a changed value rewrites one line, not the frame	Dead-display detection preserved: T24 and mutant M12
O4	Wi-Fi link polled every 100 ms; watchdog fed every 500 ms; UART poll only when the buffer is not empty	Removes per-millisecond calls that gain nothing	Link loss detected up to 100 ms later (accepted); T17 and T28 check the watchdog

7.1 Behaviour Unchanged

An optimisation that changes behaviour is a bug. The benchmark records every change of alert mask, LED and system state with its millisecond timestamp and hashes the sequence. **Week 2 and Week 3 produce the identical fingerprint (7f17efaf2e1ade28)** over one simulated hour, with identical numbers of DHT22 reads (1800), MQ-135 reads (3600), telemetry messages (238) and Wi-Fi attempts (4). The 32 tests and the soak test also pass.

8. Results

Table 8: Before and after (Week 2 baseline versus Week 3), all from the simulation

Metric	Week 2	Week 3	Change
Firmware instructions, 10 min, normal -O2 build	101.3 M	52.6 M	-48%
Whole benchmark program, 10 min, normal -O2 build	114.1 M	70.0 M	-39%
Timer ISR instructions (no-inline profile)	20.4 M	2.4 M	-88%
state_update() instructions (no-inline profile)	36.4 M	dirty-gated (not called in the benchmark)	-100%
I2C bytes per simulated hour	1,753,088	376,704	-78.5%

Metric	Week 2	Week 3	Change
I2C bus time per hour (analytical, 400 kHz)	39.4 s (1.10%)	8.5 s (0.24%)	-78.5%
I2C bytes per hour when built at -O0 (defect D1)	3,686,400	376,704	-89.8%
Interrupt-shared variables written by the timer ISR	tick, 4 flags, 2 blink phases	tick only	-6
Firmware state object (sizeof fw_t)	488 B	504 B	+16 B
Behaviour fingerprint	7f17efaf2e1ade28	7f17efaf2e1ade28	identical

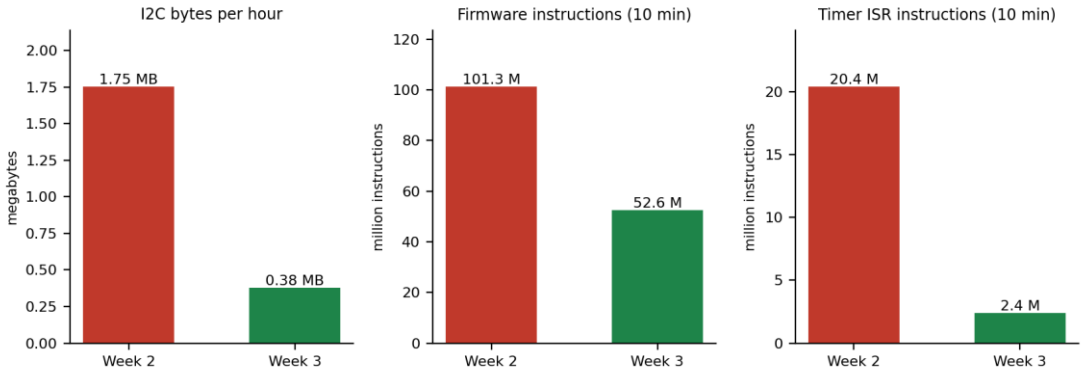


Figure 1: Measured improvements (simulation, host instruction counts and byte counts)

9. Verification Summary

Table 9: Verification of the final firmware (v3.0.0)

Check	Result
Self-test suite	32 of 32 tests, 163 checks, 0 failed
Optimisation levels -O0, -O2, -O3, -Os	All pass, 0 warnings with -Wall -Wextra -Wpedantic -Wshadow -Wconversion
AddressSanitizer and UBSan	No reports
Valgrind memcheck	0 errors (baseline: 12606 errors)
cppcheck	0 findings (baseline: style findings)
Line coverage of firmware code	99.3% (404 of 407 lines)
Mutation testing	16 of 16 injected defects caught
UART fuzzing	100000 random bytes in the test suite, no invariant violated
Soak, 24 simulated hours	0 violations, 0 watchdog resets
Soak, 7 simulated days (168 h)	0 violations; 2985 random events, 48 injected resets, 10601 telemetry messages, 0 watchdog resets

New and changed tests: T20 (display traffic), T21 and T22 (clock rollover), T23 (SET during alert), T24 (display removed mid-run), T25 (remaining UART commands), T26 (frame determinism), T27 (ISR touches only the tick), T28 (scheduler accuracy), T29 (fuzzing), T30 (worst-case message), T31 (soak), T32 (alert cleared when its sensor fails). T01 to T19 pass unchanged except for strengthened checks in T05, T09, T16 and T19.

10. Limitations

- **Simulation only.** Instruction counts are x86 host counts and show relative work. ESP32 cycle counts, interrupt latency, real sensor timing and current draw were not measured.
- **Analytical bus time.** I2C time is bytes x 9 bit times at 400 kHz, without start/stop overhead or clock stretching.

- **Interrupt races are still untested.** The simulation is single-threaded. Removing the shared task flags (O1) reduces the exposure, but the remaining ISR-shared data (tick, UART ring buffer) must be reviewed on hardware.
- **Scenario dependence.** The traffic saving depends on how often values change; a quieter or busier room changes the percentage.
- **Wi-Fi link loss** is now detected up to 100 ms later than in Week 2.
- **Thresholds and filters** are still initial proposals; the spike-filter limits and averaging window need real sensor data.
- **Soak coverage** is limited to the fault types the harness can inject.

11. Preparation for Week 4

- Run the same firmware logic through a real HAL on hardware (or an equivalent hardware-in-the-loop setup) and repeat the checks that matter: loop timing, ISR duration, current draw, I2C waveform.
- Test the interrupt-to-main data path (tick and UART ring buffer) on the target.
- Tune thresholds, spike limits and the MQ-135 warm-up with real data.
- Integrate all subsystems and run an extended unattended test against the Week 1 requirements.

12. Conclusion

Week 3 found five defects in firmware that had passed all 20 of its tests. The most important lessons were not in the firmware but in the method: a defect that disappears at one optimisation level (D1), a defect that needs 49.7 days to appear (D2), and tests that passed even when the defect was present until mutation testing exposed them. After the fixes and six rounds of test strengthening, the final firmware passes 32 tests and a 7-day randomised fault-injection run, with no memory errors reported by Valgrind.

The optimisations cut the firmware's instruction count by about half and the display bus traffic by 78%, while the hash of every alert, LED and state change is identical to Week 2. These results come from a simulation and show relative improvement; Week 4 must confirm them on real hardware.

Appendix A: Self-Test Output (final run)

```
T01  Start-up sequence, safe outputs, warm-up       PASS
T02  Normal operation and OLED content              PASS
T03  Temperature alert and response time            PASS
T04  Hysteresis (no chatter near threshold)         PASS
T05  Isolated over-limit sample ignored            PASS
T06  Humidity and low-temperature alerts           PASS
T07  Air-quality alert, recovery                   PASS
T08  Warm-up suppresses air-quality alert          PASS
T09  DHT22 disconnected, recovery                  PASS
T10  MQ-135 disconnected, recovery                 PASS
T11  OLED absent at start, later recovery          PASS
T12  Wi-Fi unavailable, back-off, reconnect        PASS
T13  Wi-Fi drops during operation                  PASS
T14  Telemetry interval and payload                PASS
T15  Transmission failure handling                 PASS
T16  UART commands and ring buffer                 PASS
T17  Watchdog feeding and lock-up reset            PASS
T18  Power interruption (brown-out) restart        PASS
T19  Spike and out-of-range data handling          PASS
T20  Display bus traffic (per-line writes)         PASS
T21  Clock rollover: state and air alert           PASS
T22  Clock rollover: periodic task timing          PASS
T23  Threshold change keeps an active alert        PASS
T24  OLED removed during operation                 PASS
T25  UART: remaining commands and limits           PASS
T26  Frame content is deterministic                PASS
T27  Timer ISR touches only the tick               PASS
T28  Scheduler accuracy (no drift)                 PASS
T29  UART fuzzing (100000 bytes)                   PASS
T30  Worst-case telemetry length                   PASS
T31  Randomised soak, fault injection              PASS
T32  Alert cleared when its sensor fails           PASS
-------------------------------------------------------------
Tests passed: 32/32   Checks executed: 163   Checks failed: 0
```

Appendix B: Soak Test Report (168 simulated hours)

```
Soak test: 168 simulated hours, seed 20261014, 2985 random events, 48 injected resets
  telemetry messages: 10601, watchdog resets: 0
  I1 outputs OFF when idle                violations: 0
  I2 alert => LED on                      violations: 0
  I3 alert => state ALERT                 violations: 0
  I4 no temp/hum alert during DHT fault   violations: 0
  I5 no AIR alert during MQ fault         violations: 0
  I6 ring buffer indices in range         violations: 0
  I7 OLED lines terminated                violations: 0
  I8 NORMAL => no alerts                  violations: 0
  I9 threshold ordering                   violations: 0
  I10 payload within buffer               violations: 0
  I11 sequence numbers increase           violations: 0
```

Appendix C: Benchmark Output, One Simulated Hour

```
Week 2 baseline:
scenario_seconds=3600
behaviour_events=4 alert_hash=7f17efaf2e1ade28
oled_writes=1712 oled_bytes=1753088 i2c_time_ms_at_400kHz=39444.5
telemetry_messages=238 wifi_attempts=4
dht_ok=1800 mq_samples=3600
sizeof_fw_t=488

Week 3:
scenario_seconds=3600
behaviour_events=4 alert_hash=7f17efaf2e1ade28 wifi_state_changes=8
oled_writes=2516 oled_bytes=376704 i2c_time_ms_at_400kHz=8475.8
telemetry_messages=238 wifi_attempts=4
dht_ok=1800 mq_samples=3600
sizeof_fw_t=504
```

Appendix D: Valgrind on the Week 2 Baseline (excerpt)

```
Command: ./v2_dbg

Conditional jump or move depends on uninitialised value(s)
   at 0x48524EE: bcmp (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
   by 0x10BE92: display_task (smart_env_monitor_v2.c:571)
   by 0x10BE92: fw_loop (smart_env_monitor_v2.c:785)
   by 0x10BE92: sim_tick (smart_env_monitor_v2.c:935)
```

```
  by 0x10C527: sim_run_ms (smart_env_monitor_v2.c:943)
  by 0x10CC09: sim_run_s (smart_env_monitor_v2.c:944)
  by 0x10CC09: t01_startup (smart_env_monitor_v2.c:1014)
  by 0x10F066: run_tests (smart_env_monitor_v2.c:1321)
```